Machine Learning, Deep Learning & NLP Applications

WEEK 1

ML Basics

Foundation concepts for understanding machine learning workflows.

- **Day 1** Introduction to Machine Learning
- **Day 2** Train / Test Split
- **Day 3** Linear Regression
- **Day 4** Classification Basics
- **Day 5** K-Nearest Neighbors (KNN)

DAY 1

Definition of Machine Learning

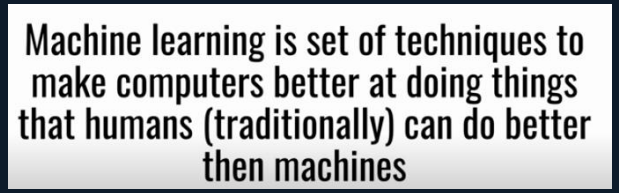
[Machine Learning is the field of study that gives computers the ability to learn without being explicitly programmed.
Arthur Samuel, 1959

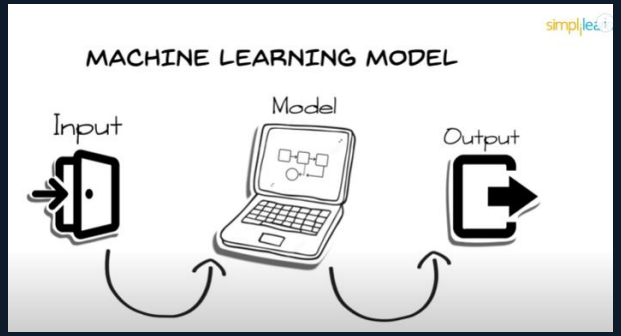
Tom Mitchell's E-T-P

“A computer program is said to learn from experience E with respect to some task T and some performance measure P, if its performance on T, as measured by P, improves with experience E.” -Tom Mitchell (1999)

The Mapping Function

ML represents a mapping f: X → Y trained to minimize empirical loss on historical data under constraints like latency and interpretability. Engineers must align training objectives with business KPIs.





DAY 1

What ML Is NOT

Not a Shortcut

ML is not a shortcut to insight when labels are ambiguous, nor a guarantee that raw data will reveal value

Not Always Needed

Problems solvable with deterministic rules or lookup tables gain little from ML while inheriting stochastic failure modes and compliance risk

Warning Signs

Naive practitioners overfit dashboards

Throwing algorithms at every dataset

Ignoring simpler solutions

DAY 1

Learning Paradigms

Supervised

In supervised learning, the training data you feed to the algorithm includes the desired solutions, called labels

Unsupervised

In unsupervised learning, the training data is unlabeled. The system tries to learn without a teacher.

Example of unsupervised learning tasks are:

clustering

anomaly detection

Reinforcement

The learning system (agent) observe the environment, select and perform actions, and get rewards in return (or penalties in the form of negative rewards) .

It must then learn by itself what is the best strategy(policy) to get the most reward over time. A policy defines what action the agent should choose when it is in a given situation.

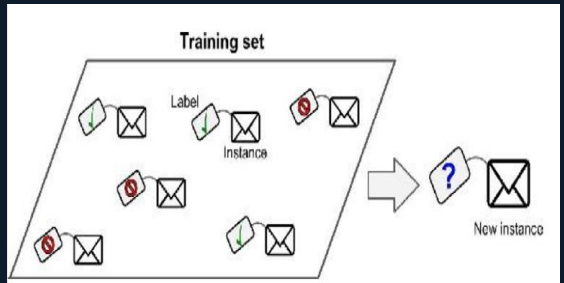
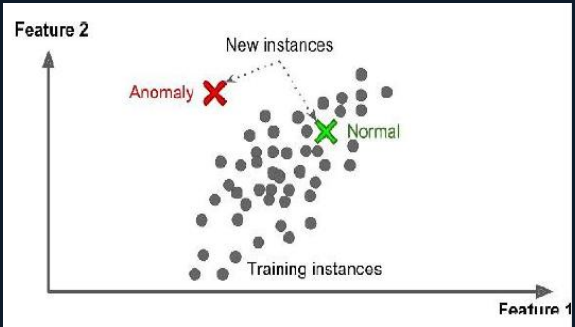


Figure 1-5. A labeled training set for supervised learning (e.g., spam classification)



WEEK 1 · KEY CONCEPTS

Supervised vs Unsupervised Learning

Supervised Learning

SUPERVISED

Has labels: "This is a cat" / "This is a dog"

UNSUPERVISED

No labels: "Find patterns yourself"

Model learns from labeled examples. Given input-output pairs, it learns the mapping.

DAY 1

Fields of Application of Machine Learning

Computer Vision

Computer vision is an interdisciplinary scientific field that deals with how computers can gain high-level understanding from digital images or videos.

Natural Language Processing

Natural language processing (NLP) is a branch of artificial intelligence that helps computers understand, interpret and manipulate human language.

Time Series Prediction

Time series prediction is the use of a model to predict future values based on previously observed values.

1-Bitcoin Price Prediction Using Time Series Forecasting

2-Stock price prediction

WEEK 1 DAY 1

The Machine Learning Workflow

Overview

End-to-End ML Pipeline

1 DATA -> 2 MODEL -> 3 PREDICT -> 4 EVALUATE

Every ML project follows this fundamental cycle: collect data, train a model, make predictions, and measure performance

WEEK 1 DAY 1

Step 1: Data Collection & Preparation

Data Sources

- Databases (SQL, NoSQL)
- APIs and Web Scraping
- CSV/Excel Files
- Sensors and IoT Devices
- Public Datasets (Kaggle, UCI)

Data Preparation Tasks

- Handle missing values
- Remove duplicates
- Fix data types
- Normalize/Scale features
- Encode categorical variables

KEY INSIGHT

Data quality determines model quality. 80% of ML work is data preparation. Garbage in = garbage out.

WEEK 1 DAY 1

Step 2: Model Training

What Happens During Training?

1. Initialize Parameters
Model starts with random weights

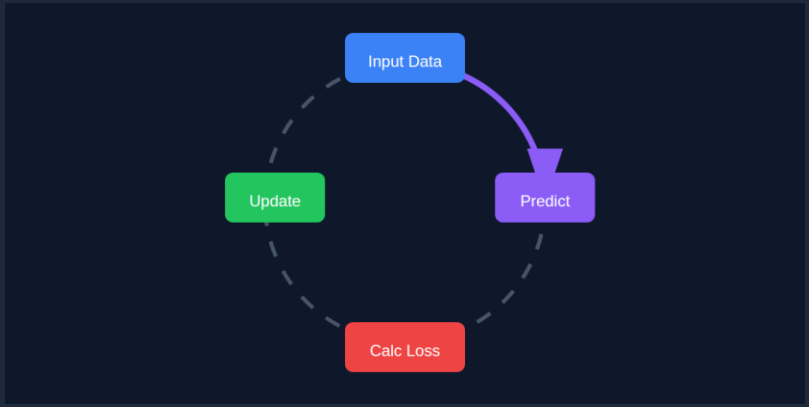
2. Make Predictions
Forward pass through the model

3. Calculate Error (Loss)
Compare predictions to actual values

4. Update Parameters
Adjust weights to reduce error

5. Repeat
Iterate until convergence

Training Loop Visualization



WEEK 1 DAY 1

Step 2: Choosing a Model

Different problems require different algorithms. Here are the main categories:

Classification

Predict a category/class

- Logistic Regression
- Decision Trees
- Random Forest
- SVM
- Neural Networks

Example: Spam detection, Disease diagnosis

Regression

Predict a continuous value

- Linear Regression
- Polynomial Regression
- Ridge/Lasso
- Gradient Boosting
- Neural Networks

Example: House prices, Stock prediction

Clustering

Group similar items

- K-Means
- Hierarchical
- DBSCAN
- Gaussian Mixture
- Spectral Clustering

Example: Customer segments, Anomalies

WEEK 1 DAY 1

Step 3: Making Predictions

From Input to Output

INPUT FEATURES

Age: 35
Income: $75,000
Credit Score: 720
Debt Ratio: 0.3

->

TRAINED MODEL

Learned patterns from 10,000 examples

->

PREDICTION

APPROVED

Confidence: 87%

Batch Prediction

Process many examples at once. Used for periodic scoring, reports, or offline analysis.

Real-time Prediction

Instant predictions via API. Used for live applications like fraud detection or recommendations.

Probability Output

Many models output confidence scores, allowing you to set custom thresholds for decisions.

WEEK 1 DAY 1

Step 4: Evaluation › Measuring Success

How do we know if our model is good? We measure its performance using metrics:

Classification Metrics

Accuracy 92%
Correct predictions / Total predictions

Precision 89%
True positives / Predicted positives

Recall 94%
True positives / Actual positives

F1 Score 91%
Harmonic mean of precision and recall

Regression Metrics

MAE 14
Mean Absolute Error › avg difference

MSE 8.2M
Mean Squared Error › penalizes large errors

RMSE 300
Root MSE › same units as target

R^2 Score 0.87
Variance explained (0-1, higher is better)

WEEK 1 DAY 1

ML Workflow › Complete Summary

1. DATA

Collect, clean, explore
Split into train/val/test
Feature engineering

~60-80% of project time

2. MODEL

Choose algorithm
Train on training data
Tune hyperparameters

Learn patterns from data

3. PREDICT

Apply to new data
Batch or real-time
Get probabilities

Generate insights/decisions

4. EVALUATE

Measure performance
Analyze errors
Iterate and improve

Continuous improvement

KEY TAKEAWAYS

Data quality > Model complexity

Always evaluate on held-out data

Iterate based on error analysis

DAY 1

Loss, Optimization & Generalization

Loss Functions

Quantifies how far predictions are from the true values.

Translates model error into a number the model can learn from.

Lower loss means predictions better match the data.

Optimization

The process of updating model parameters to reduce loss.

Uses gradient-based methods to move toward better solutions.

Learning rate controls how fast or stable learning is.

Generalization

How well the model performs on new, unseen data.

A model can minimize training loss but still fail to generalize.

Good generalization means learning patterns, not memorizing data.

DAY 1

Misconceptions & Dataset Realities

More Features ≠ Better

Having more features (columns) does not automatically make a model better. Extra or repeated information can confuse the model and make predictions worse. Choosing the right features is more important than using many.

Missing Data Encodes Bias

Missing values are not random. They often happen for real reasons, such as access, income, or reporting issues. If we ignore why data is missing, the model can learn unfair or incorrect patterns.

Inspection Required

Before building a model, you must examine the dataset. Check if some groups are over- or under-represented, if information accidentally reveals the answer, or if the data was collected in a biased way.

DAY 1 · ACTIVITY

ML Foundations Assessment

ML Foundations Activity (Link)

- Define ML using the E-T-P framework and identify the input–output mapping.
- Distinguish supervised, unsupervised, and reinforcement learning signals with mini examples.
- Train a small classifier, compare accuracy vs loss, and reflect on generalization.

DAY 2

Train / Test Discipline

Safeguarding credibility by separating learning signals from evaluation signals so reported metrics predict future performance

DAY 2

Learning Objectives

- Explain why unseen evaluations prevent inflated claims
- Differentiate training, validation, and test subsets for iterative tuning
- Detect overfitting and data leakage visually and analytically
- Implement train_test_split with stratification and fixed random states

DAY 2

Why Evaluate on Unseen Data

The Analogy

Testing a model on the same data it learned from is like testing a product only with the people who built it. It may look perfect, but it hasn't been tested in the real world.

The Right Approach

We set aside some data that the model never sees during training. This "unseen" data acts like real-world use and shows how the model will behave after deployment.

Why This Matters

Models can appear very accurate simply because they memorized the training data. When used in real situations, these models often fail, leading to poor decisions, financial losses, or safety risks.

DAY 2

Training vs Testing Roles

Training Set

This is the data the model learns from. It is used to adjust the model's internal settings so it can make predictions.

Validation Set

This data is used while building the model to make decisions, such as choosing settings or improving performance. The model does not learn directly from this data.

Test Set

This data is kept completely separate and is only used at the very end to measure how well the model performs on new, unseen data.

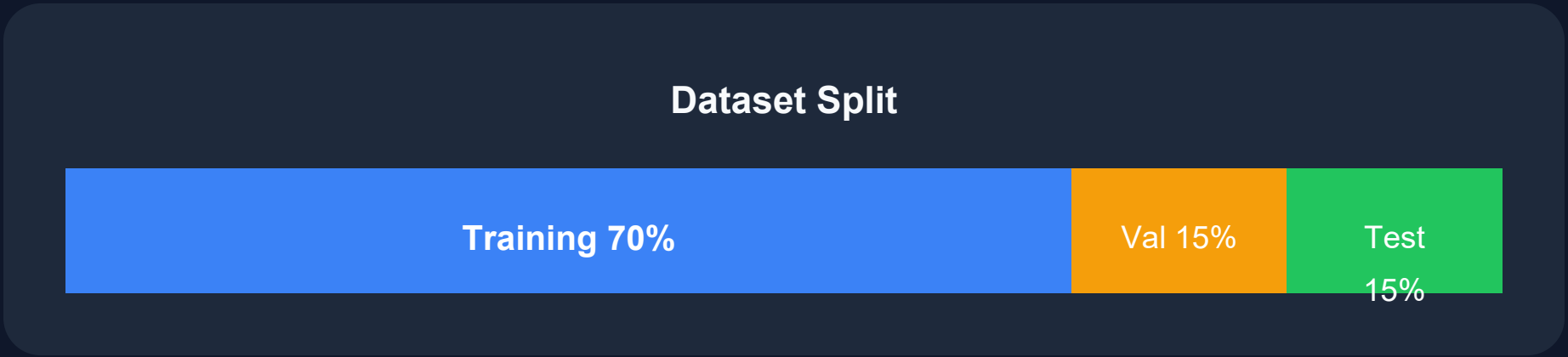
Common Mistake

X

Beginners often look at test results during development to fix issues. Doing this turns the test set into extra training help and makes the final performance results unreliable.

DAY 2

Train/Test Split



DAY 2

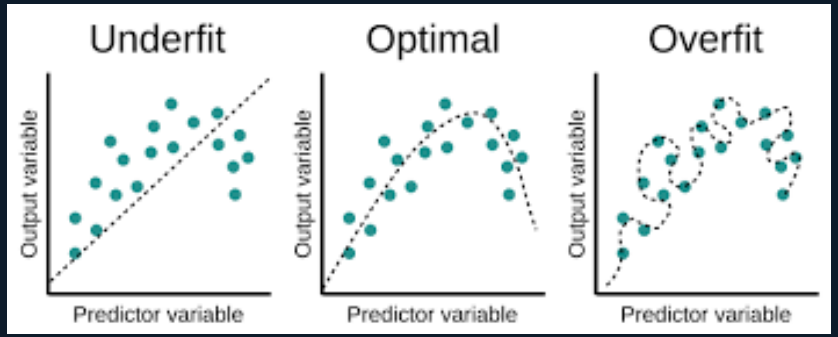
Overfitting & Underfitting Detection

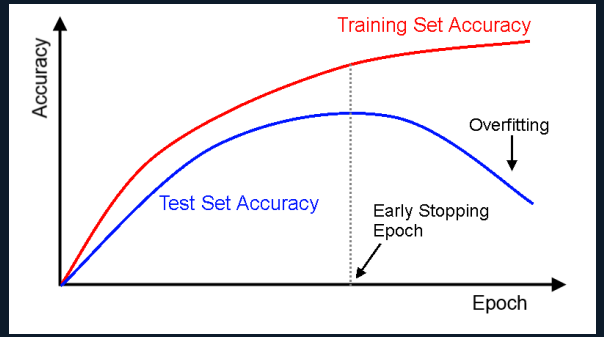
Overfitting

High training accuracy + poor test accuracy. Model encodes noise or identifiers instead of patterns.

Underfitting

model cannot capture the underlying trend of the training data.





DAY 2

Data Leakage Hazards

What Is Data Leakage?

Data leakage occurs when a model learns information it would not have in real-world use, making results misleading.

Feature Leakage

Including future or outcome-related information (e.g., “days since default”) gives the model the answer in advance.

Preprocessing Leakage

Calculating statistics on the full dataset before splitting allows test data to influence training.

Why It Matters

Leakage can produce near-perfect test accuracy but cause models to fail badly in production, which is why strict safeguards are now required.

DAY 2

Random State & Stratification

random_state

Sets a fixed seed so data splits are reproducible. This allows others to get the exact same results and makes experiments reliable.

stratify = y

Ensures class proportions stay the same across splits, so rare cases (like fraud) are not accidentally excluded from the test set.

Without These

Each split is different, causing large swings in metrics and making results hard to trust or compare.

Each split is different, causing large swings in metrics and making results hard to trust or compare.

DAY 2

Split Ratio Trade-Offs

More Training Data

Gives the model more examples to learn from, which can improve learning, but leaves less data to properly evaluate performance.

More Test Data

Provides a more reliable measure of real-world performance, but reduces the amount of data available for learning.

Small Datasets

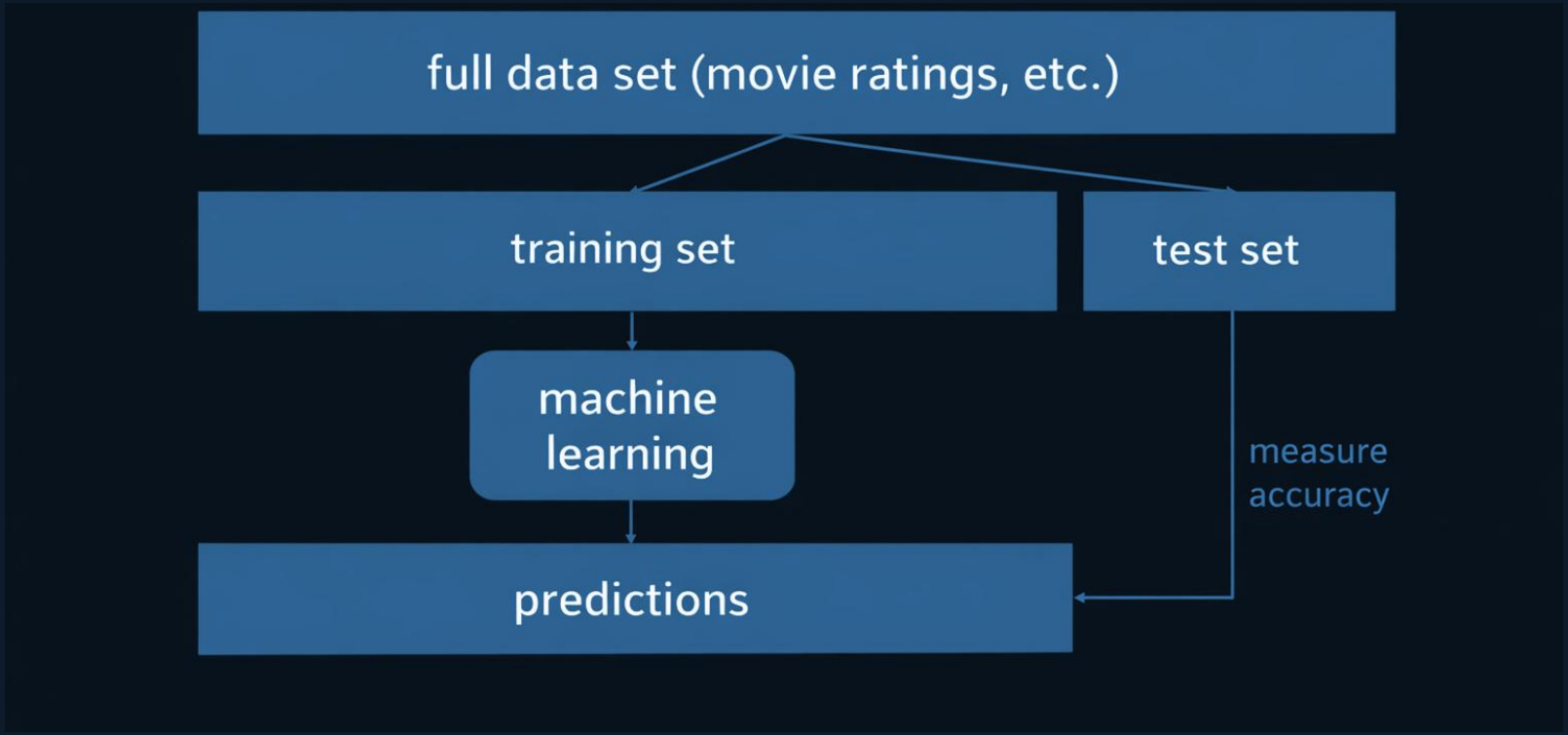
When data is limited, k-fold cross-validation is often used to get more stable and trustworthy performance estimates.

How to Choose

The best split depends on:

- How much data you have
- How reliable the evaluation needs to be (e.g., regulatory requirements)
- Available computing resources

Train/Test Split



DAY 2

Pipeline & Reproducibility

The Problem

If you clean, scale, or fill missing values before splitting the data, the model accidentally uses information from the entire dataset. This makes performance look better than it really is.

Best Practice

Put all preprocessing steps into a pipeline and fit them only on the training data. This prevents information from leaking into testing.

Version Control

Save both the data version and the code used to split it. This allows results to be reproduced later and verified by teammates or auditors.

DAY 2 · ACTIVITY

Train/Test Split Assessment

Train/Test Split (Link)

- Train a baseline model without a split to see why results can be misleading.
- Apply a proper train/test split with stratification and evaluate accuracy.
- Demonstrate data leakage and show the correct preprocessing workflow.

DAY

3

Linear Regression Baselines

Constructing interpretable continuous predictors while understanding when linear assumptions succeed or crumble

DAY 3

Learning Objectives

- Classify problems as regression tasks based on target type
- Articulate linear model equations with biases and coefficients
- Train Linear Regression models and compute MSE/RMSE/R^2
- Interpret coefficients in units meaningful to stakeholders
- Critique limitations such as heteroscedasticity or multicollinearity

DAY 3

Linear Regression

What You Will Learn

1. Understand what linear regression does
2. Learn the equation and what each part means
3. Build your first model with scikit-learn

The Big Idea

Linear regression finds the best straight line through your data points. Once you have this line, you can predict new values.

Example: Predict house price from square footage

DAY 3

Defining Regression Problems

Regression Targets

Regression is used when the goal is to predict a number that can take many possible values, such as price, temperature, or energy usage.

Common Mistake

Using category labels or ID numbers (like product IDs or zip codes) as regression targets leads to incorrect results. The model treats numeric differences as distances, even when they have no real meaning.

Quick Check

- ❑ Is the target truly continuous?
- ❑ Do differences between values have meaning?
- ❑ Would averaging make sense?

DAY 3

The Linear Regression Equation

$$y = mx + b$$

y
The output you want to predict
Example: House price

m
Slope: how much y changes per unit of x
Example: $200 per sq ft

x
The input feature you use
Example: Square footage

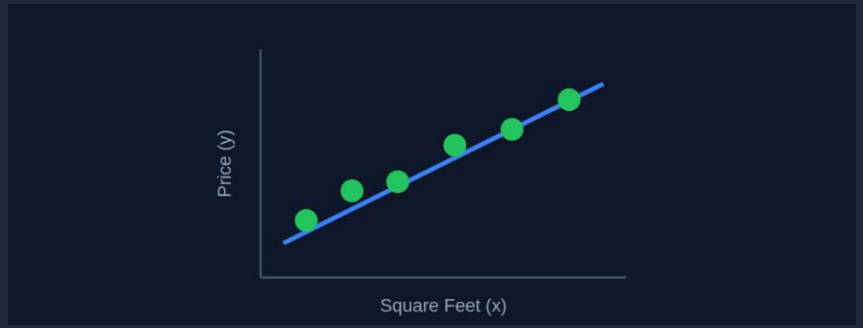
b
Intercept: starting value when x=0
Example: $50,000 base

Real Example: price = $200 × sqft + $50,000 → A 1000 sqft home = $200 × 1000 + $50,000 = $250,000

DAY 3

How It Works Visually

Finding the Best Fit Line



Data points Best fit line

Slope (m) Controls the Angle

m > 0	m < 0	m = 0
Line goes UP	Line goes DOWN	Flat line

Intercept (b) Shifts Up/Down

Where the line crosses the y-axis. Higher b = line shifts up. Lower b = line shifts down.

DAY 3

How the Model Finds the Best Line

Step 1: Measure Errors

For each point, calculate how far the prediction is from the actual value.

Red lines = errors (residuals)

Step 2: Calculate MSE

$$MSE = (1/n)\ \Sigma(y - \hat{y})^2$$

Square each error, then average them. Squaring makes all errors positive and penalizes big errors more.

Step 3: Minimize

The algorithm adjusts m and b to make MSE as small as possible.

Lowest MSE = Best Line!

The model tries thousands of different m and b values to find the combination with the smallest total error.

DAY 3

Building Your First Model with Scikit-Learn

Complete Code Example

```
# 1. Import the tools
from sklearn.linear_model import LinearRegression
from sklearn.model_selection import train_test_split

# 2. Split data into train/test
X_train, X_test, y_train, y_test = train_test_split(X, y)

# 3. Create and train the model
model = LinearRegression()
model.fit(X_train, y_train)

# 4. Make predictions
predictions = model.predict(X_test)
```

Key Methods

.fit(X, y)
Trains the model

.predict(X)
Makes predictions

Get the Equation

```
m = model.coef_[0]
b = model.intercept_
```

DAY 3

Evaluating Your Model

Common Metrics for Regression

R^2 Score — 0.0 to 1.0

How much variance your model explains. 1.0 = perfect, 0 = useless

MAE (Mean Absolute Error) — Same units as y

Average error in original units. Easy to interpret.

RMSE (Root Mean Squared Error) — Same units as y

Like MAE but penalizes large errors more heavily.

Code to Evaluate

```
from sklearn.metrics import r2_score, mean_absolute_error

r2 = r2_score(y_test, predictions)
mae = mean_absolute_error(y_test, predictions)

# Or use built-in score
r2 = model.score(X_test, y_test)
```

Tip: Always evaluate on TEST data

Never on training data!

DAY 3

Day 3 Summary

The Equation

$$y = mx + b$$

m = slope (rate of change)
b = intercept (starting point)

The Goal

Minimize MSE

Find the line that minimizes squared errors between predictions and actual values.

The Code

```
model = LinearRegression()
model.fit(X_train, y_train)
pred = model.predict(X_test)
```

DAY 3 · ACTIVITY

Linear Regression Assessment

Linear Regression (Link)

- Train a linear regression model on study hours vs final score.
- Report model coefficients and evaluate error with MAE/RMSE.
- Plot the regression line to visualize the trend.

Predicting Home Price in Monroe, NJ (Link)

- Fit a linear regression model to predict home prices from square footage.
- Make price predictions for new areas and inspect model coefficients.
- Generate a CSV of predicted prices for a list of areas.

DAY 4

Classification Basics

Mapping inputs to discrete labels and ensuring evaluation respects domain costs rather than surface-level accuracy

DAY 4

Learning Objectives

- Define classification problems and distinguish binary vs multi-class
- Contrast classification losses with regression losses
- Implement a scikit-learn classification pipeline
- Compute accuracy, precision, recall, and F1 scores
- Adjust thresholds when errors have asymmetric costs

DAY 4

What Classification Looks Like

Classification Targets

Predicts membership in categories: churn vs retain, species A/B/C, spam vs not spam

Common Mistake

Treating ordinal labels numerically—class '2' is not twice class '1'

Examples

Email spam detection

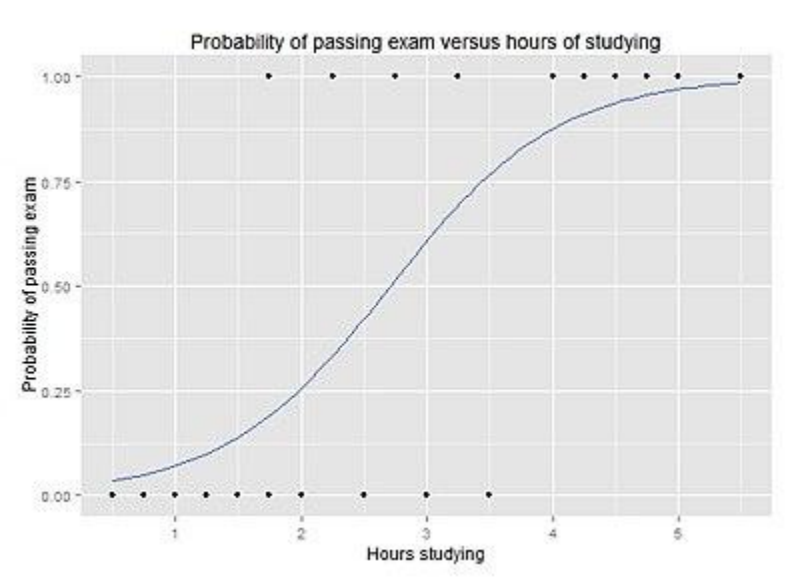
Medical diagnosis

Customer churn prediction

Classification using Logistic Regression

Example: Probability of passing an exam versus hours of study

Hours	0.50	0.75	1.00	1.25	1.50	1.75	1.75	2.00	2.25	2.50	2.75	3.00	3.25	3.50	4.00	4.25	4.50	4.75	5.00	5.50
Pass	0	0	0	0	0	0	1	0	1	0	1	0	1	0	1	1	1	1	1	1



DAY 4

Binary vs Multi-Class

Binary Classification

Single threshold, straightforward probabilistic outputs. Confusion matrix is 2×2.

Multi-Class

Requires one-vs-rest or multinomial formulations. Confusion matrices expand correspondingly.

Key Insight

Per-class precision/recall essential

No class should be neglected

Aggregate accuracy can hide problems

DAY 4

Regression vs Classification

Don't Use Regression for Classes

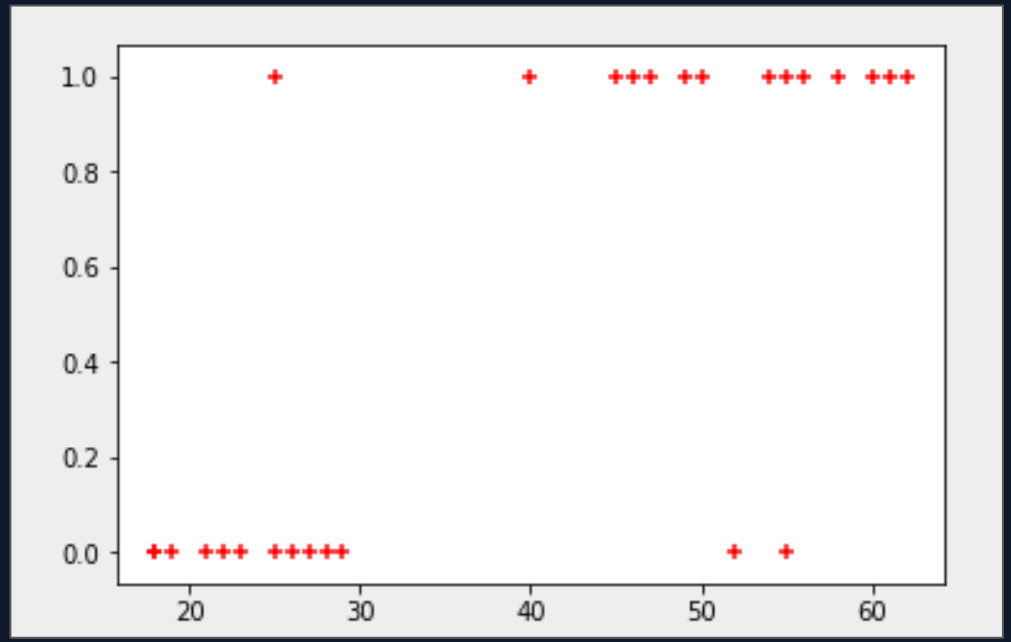
Fitting regression on class IDs makes distances meaningless—optimizer averages label numbers producing fractional nonsense

Classification Uses Cross-Entropy

Aligns predicted probabilities with true labels, ensuring outputs are class likelihoods

Example: Prediction if a customer will buy an insurance or not

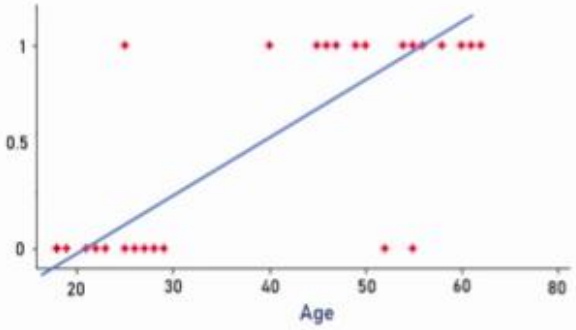
age	have_insurance
22	0
25	0
47	1
52	0
46	1
56	1
55	0
60	1
62	1
61	1
18	0
28	0
27	0
29	0
49	1

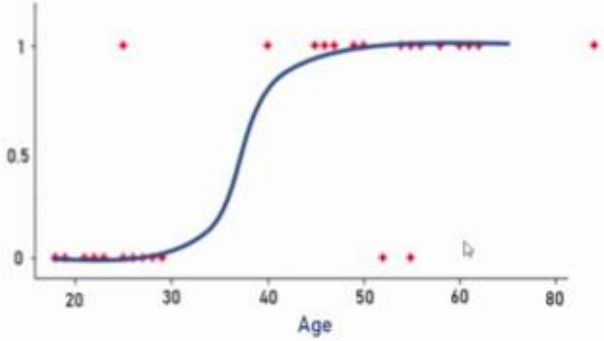


Example: Prediction if a customer will buy an insurance or not

$$y = m * x + b$$

$$y = \frac{1}{1 + e^{-(m*x+b)}}$$





DAY 4

Probabilities, Thresholds & Costs

Default Threshold = 0.5

Students default to 0.5, but this ignores business context

Fraud Detection Example

Lowering threshold raises recall (catches more fraud) but increases false positives

Best Practice

Explicit threshold tuning tied to business cost matrices, not defaults

DAY 4

Confusion Matrix & Metrics

TP
True positive

FP
False positive

FN
False Negative

TN
True Negative

Imbalanced Data Warning

1% positives: predicting all negatives yields 99% accuracy but zero recall

DAY 4

Risks, Misconceptions & Fairness

Accuracy Isn't Enough

Ignores fairness constraints and user trust. High aggregate metrics can hide subgroup failures.

Biased Training Data

Models systematically misclassify minority groups even with high overall metrics

Required Checks

Subgroup analysis, fairness-aware reweighting, and human oversight prevent regulatory backlash

DAY 4 · ACTIVITY

Classification Basics Assessment

Classification Basics (Link)

- Build a logistic regression classifier on a synthetic binary dataset.
- Train/test split for reproducibility and inspect model coefficients.
- Evaluate accuracy and confusion matrix to interpret performance.

Logistic Regression Fraud Detection (Link)

- Load a real-world fraud dataset and one-hot encode categorical features.
- Train a logistic regression model with a train/test split.
- Evaluate performance with accuracy and a confusion matrix.

DAY

5

K-Nearest Neighbors

Instance-based learning that forces geometric reasoning about feature space, scaling, and the bias–variance trade-off

DAY 5

Learning Objectives

- Articulate KNN intuition and majority voting mechanism
- Explain why scaling and distance metrics dictate behavior
- Analyze the influence of K on bias and variance
- Implement KNeighborsClassifier and compare performance across Ks
- Discuss scenarios where KNN excels or fails

DAY 5

Intuition Behind KNN

The Algorithm

KNN stores all labeled points and predicts via majority vote among K closest neighbors. No explicit training occurs.

Transparency Benefit

"These specific neighbors influenced this decision"—offers local interpretability

The Risk

Noise or mislabeled points severely distort predictions, especially in sparse regions

DAY 5

Distance Metrics & Scaling

Euclidean

Best for continuous features

Manhattan

Best for grid-like data

Cosine

Best for high-dimensional embeddings

Critical: Always Scale

Raw units (income in dollars vs age in years) dominate distances. Use StandardScaler before KNN.

DAY 5

Role of Feature Engineering

What's the Problem?

Using too many features can confuse the model instead of helping it.

Why KNN Struggles

KNN decides based on distance. Extra or meaningless features change distances and cause the model to pick the wrong neighbors.

Easy Example

Imagine comparing houses using:

Size (square feet) – useful Number of bedrooms – useful House ID number – not useful

What Helps

- Keep only useful features
- Remove features that don't matter
- Reduce the number of features when possible
- Use better representations for complex data (like text or images)

DAY 5

Choosing K: Bias-Variance Trade-Off

K=1

Low bias, high variance. Reacts strongly to noise.

K=5

Often a balanced sweet spot for many datasets

K=15

High bias, low variance. May erase minority class structure.

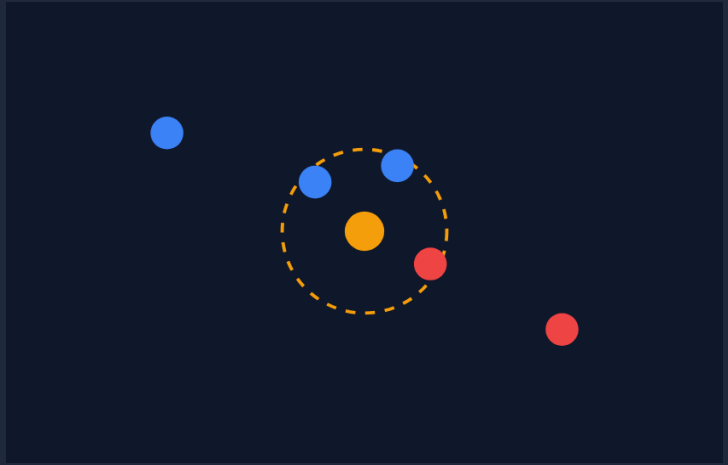
Finding the Sweet Spot

Plot validation accuracy vs K to reveal optimal values—both extremes carry risk

DAY 5

KNN Classification

K-Nearest Neighbors



Query point finds K nearest neighbors

DAY 5

Effect of K on Decision Boundaries

K=1 Boundaries

Jagged, irregular edges that closely follow individual training points

Large K Boundaries

Overly smoothed regions that may misalign with true class structure

Business Decision

Selecting K ties to tolerance for false positives vs false negatives and sensitivity to local anomalies

DAY 5

Handling Noise & Class Imbalance

Noisy Labels

Some data points are labeled incorrectly. When these points are near class boundaries, KNN can be easily misled and make wrong predictions.

Class Imbalance

When one class has many more examples than another, KNN tends to favor the larger class.

Key Insight

KNN doesn't automatically respect minority structure

Explicit handling required

DAY 5 · ACTIVITY

KNN Classification Assessment

KNN Classification: Choosing K (Link)

- Train KNN classifiers with multiple K values and compare accuracy.
- Identify the best K and check class balance in train/test splits.
- Highlight the K-too-large pitfall and apply a safe K choice.

Diabetes KNN (Link)

- Load the diabetes dataset and train a KNN model to evaluate accuracy and F1.
- Clean and preprocess data (handle zeros, scale features) to improve performance.
- Tune K and compare metrics before and after preprocessing.